

Fees and Receipt Service: Final HandsOn Project

Fees and Receipt Service

A Spring Boot API for managing student fee invoices, payments, and receipts – built to be correct under retries, correct under concurrent access, and auditable after the fact.

What This Project Does

Schools and training institutes need to track what students owe and what they've paid. Done manually or with a naive system, this goes wrong in predictable ways: a student gets charged twice because a payment request was accidentally sent twice, invoice totals don't match what was actually billed, or nobody can tell whether a "successful" payment actually cleared with the payment processor.

This project is a deliberately correct, small-scale billing ledger that closes off each of those failure modes by design, not by hoping they don't happen.

Key Capabilities

- **Students and invoices:** create students, issue itemized invoices, with totals always calculated from the actual line items rather than trusted from whatever a client sends.
- **Idempotent payments:** The same payment request, sent twice for any reason (a network retry, a nervous double-click), is guaranteed to never create a duplicate charge.
- **Enforced financial rules:** a payment can't exceed an invoice's outstanding balance, can't be recorded in the wrong currency, and can't be applied to an invoice that isn't in a payable state.
- **Computed balances:** Student and invoice balances are calculated fresh on every request, so they can never silently drift out of sync with reality.
- **PDF receipts:** a downloadable, itemized receipt generated on demand for any payment.
- **Reconciliation reporting:** Flags which payments have been independently confirmed by the payment gateway versus which are still awaiting confirmation.
- **Mock payment gateway webhook:** A signed, verifiable callback mechanism simulating how a real payment gateway (e.g. Stripe, Paystack) would confirm a transaction after the fact.

Technology Stack

Area	Choice
Framework	Spring Boot 3 (Java 21)
Database	PostgreSQL, via Spring Data JPA / Hibernate
Migrations	Flyway
Receipts	Thymeleaf + Flying Saucer (PDF rendering)

API Surface

Resource	Endpoints
Students	Create, fetch, view balance
Invoices	Create, fetch, issue, view balance
Payments	Record (idempotent), fetch, download receipt
Reports	Reconciliation report by date range
Webhooks	Gateway payment confirmation (HMAC-verified)

Full interactive API documentation is available via Swagger UI once the app is running.

Running It Locally

1. Copy `application-local.example.yml` to `application-local.yml` and set a real database password.
2. Run with the local profile: `mvn spring-boot:run "-Dspring-boot.run.profiles=local"`
3. Swagger UI: <http://localhost:8080/swagger-ui.html>

Project Status

All core functionality is complete and manually tested end to end: student/invoice management, idempotent payments, balance calculation, PDF receipts, reconciliation reporting, and mock webhook confirmation. Automated integration tests cover the full payment flow, including a regression test for a currency-validation bug caught during development.

Visit the `README.md` file for a detailed breakdown on every part of the project.